

878 lines (683 loc) · 32.9 KB

Preview Code Blame

🔗 Sentinel — architecture and build plan

Status: DESIGN ONLY. Nothing in this document is implemented. Stocker is being retired in favour of Sentinel, a much smaller deterministic trading appliance built from Stocker's proven parts. This file is written so that a context with no memory of the conversation that produced it can pick up the build.

Read this before `docs/wealth-core-certification.md` if you are new; read the certification manifest first if you are continuing the Wealth Core rehearsal, which is unrelated work still in flight.

🔗 1. What Sentinel is

Sentinel is not a stock-picking strategy. It is a deterministic risk and exposure controller wrapped around Wealth Core.

Wealth Core remains the alpha engine and the authoritative book. Sentinel only decides **how much** of the live account is exposed to Wealth Core versus a defensive Treasury-bill sleeve. It never decides **what** Wealth Core holds.

Market/session data



|
v

Wealth Core <- authoritative strategy state, ALWAYS
fully invested

|
v

Immutable shadow <- the same book, never de-risked; the
truth source

|
v

Sentinel features <- drawdown, breadth, momentum, vol, all
from the SHADOW

|
v

Sentinel state machine <- fast + slow severe causes, tracked
separately

|
v

Target exposure <- 0.0 or 1.0. Binary. Nothing in
between

|
v

Execution projection <- shadow holdings x exposure -> real
share quantities

|
v

Next-open execution <- orders, fills, cash, reconciliation

🔗 The single most important invariant

The shadow never de-risks. When the live account goes defensive, the shadow continues as if Wealth Core had remained fully invested. Every Sentinel measurement comes from the shadow, never from the de-risked live book.

Without this, the controller distorts the signals it uses to decide recovery: a book that de-risked would show a shallower drawdown, which would look like recovery, which would re-enter, which would deepen the drawdown. The shadow breaks that loop by construction.

🔗 The dependency direction, which must never invert

Wealth Core shadow -> Sentinel evidence -> live exposure



Never live book -> rebuild shadow. The shadow is authoritative; the live book is derived.

2. Why Stocker is retired

Stocker was a general-purpose quant research platform: ~16 services, an LLM evaluator in the loop, news sentiment, a strategy marketplace, multiple portfolio builders, a factor experimentation framework, and two simulators. That shape was right for exploration and is wrong for operating one frozen deterministic strategy.

Sentinel is a radically smaller problem: **one deterministic engine, two runtime roles, one database.**

Retire, do not erase

Keep the Stocker repository and history. Tag it (`stocker-legacy-2026-08`) and mine it. Before discarding any subsystem, check whether it contains one of four things worth carrying forward:

- 1 broker / execution plumbing
- 2 data ingestion and normalisation
- 3 operational UI and alerting
- 4 proven edge-case fixtures and invariants



Category 4 is the most valuable and the easiest to overlook. Stocker's invariant tests encode outages that actually happened — the `DateAnchor` re-trigger loop, `/health` performing dependency I/O, the capacity contract between planner and risk gate. Those are worth more than most of the code.

What is carried forward, essentially intact

The Wealth Core engine and everything certified around it:



```
shared/stock_strategy_shared/wealth_core/  
  run.py          the session driver; already supports  
deterministic resumption  
  adapter.py      the fixed daily event ordering (spec §11)  
  state.py        slots, episodes, peaks, cooldowns, review  
flags  
  terminal.py     corporate actions, C1/C2 settlement  
waterfall  
  settlement.py   the terminal-settlement rule, pure  
  marks.py        mark status and the resolved/estimated  
equity split  
  eligibility.py  point-in-time investability  
  feed.py         normalised vendor bars, warm-up windows  
  hashes.py       the seven parity hashes + streaming  
primitives  
  performance.py CAGR, drawdown, turnover, benchmark  
comparison  
  golden.py       the golden scenario  
  risk_profile.py wealth_core_v1
```

Plus the certification apparatus: the golden fixture, the restart matrix, the parity manifest, and `services/bt-engine/app/wealth_core_chain.py` 's rehearsal.

🔗 What is explicitly NOT carried forward

Do not rebuild these unless a need is proven:



```
LLM evaluator, LLM memory, llm-gateway, Ollama  
news sentiment, Tavily  
Alpha Vantage ingestion  
the generic regime machinery  
the factor experimentation framework  
multiple independent portfolio builders  
the strategy marketplace / registry  
microservices per pipeline stage  
duplicate simulators  
separate backtest-only strategy implementations
```

Research on new strategies happens outside the production platform. When something is worth promoting, the deterministic rule is implemented in the small core and certified.

3. Runtime architecture

image: sentinel-engine:<git-sha>	ONE artifact	
sentinel-live	mode=live	small memory limit
sentinel-bt	mode=backtest	large memory limit
postgres	one database	



Invariant: live and backtest containers may differ in configuration and runtime role, but never in engine code artifact. This makes live/backtest parity structural instead of something tests must keep re-proving — the artifact being certified is literally the artifact that executes.

Two containers, not one. Stocker learned this the expensive way: a live deploy that recreates containers mid-backtest destroys the run, and three consecutive nightly baselines were lost to exactly that before an in-flight guard was added. Separate lifecycles are not optional.

A UI can start inside `sentinel-live` and move out only if it earns its own container. Same for the broker adapter.

4. `run_until(date)` — the core capability

There is no `backtester -> live engine` split. There is one deterministic session machine with one entry point:

<code>engine.restore(snapshot)</code>	load a trusted snapshot
<code>engine.prime(history_window)</code>	rebuild rolling signal inputs;
MUTATES NOTHING else	
<code>engine.run_until(date)</code>	advance sessions



Historical certification: warm up, run 2006..2026. Production on Monday: restore Friday's snapshot, prime, process Monday.

The only difference is the source of the next session.

🔗 Warm-up has a narrow meaning

Warm-up rebuilds **rolling signal inputs** (the trailing window the ranker needs). It does **not** reconstruct path-dependent portfolio history — slots, cooldowns, episode peaks, review flags and pending actions come from the snapshot, because a 127-session price window cannot derive them.

🔗 `prime()` must use a fresh Feed by construction

```
feed = Feed.from_history(window)      # correct
engine.attach_feed(feed)

engine.feed.prime(window)              # WRONG — invites
mutation of a restored feed
```

`Feed` is stateful. Stocker's chain rehearsal already has to build a *second* warmed feed for its bulk replay, because sharing one replays the whole run through it twice and surfaces as a divergence with nothing to do with the strategy. That is the exact hazard `prime()` reopens.

Falsifier (write this test):

```
uninterrupted_run(d)
==
restore(snapshot_s) + prime(history_before_s) + run_until(d)
```

compared on: portfolio state hash, feed hash, pending-action hash, ledger hash, decision hash, candidate_audit hash, daily_state hash, daily_equity hash, final_result hash, and the exact ordered decisions and fills of the suffix.

🔗 5. Snapshots and recovery

The live book is reconstructible, not disposable. The precise contract:

The live book is reconstructible from a pinned corpus version + a persisted engine snapshot + subsequent immutable session inputs and fills.

This matters because **historical data is not immutable**. Sharadar restates; `bt_data_version` exists and gets bumped when data is corrected in place. If you replay from inception against today's corpus you may not reproduce the book you actually traded, and the divergence report cannot tell you whether the broker drifted or the history moved.

🔗 Two distinct operations

```
resume_live()  
    load latest trusted snapshot  
    verify strategy fingerprint + data_version +  
    snapshot_schema_version  
    replay only sessions after the snapshot  
    reconcile against broker  
  
audit_reconstruct()  
    pin an EXPLICIT historical corpus version  
    replay from an earlier trusted anchor or inception  
    compare hashes
```

Full-inception replay is a deliberate audit operation, **not the boot path**. Measured: 6.62M bars over 3.5 years is ~2.9 GB resident; twenty years at ~10k tickers is ~17M bars ~ 7.5 GB, more than the current NAS has.

🔗 Snapshot cadence and retention

Snapshot **every completed session** — the state is small and it minimises the recovery surface. Retain daily for 90 days, monthly indefinitely.

🔗 Snapshot schema versioning is mandatory from day one

`snapshot_schema_version` belongs **inside the state hash**, so a mismatched schema cannot produce a hash that looks comparable.

```
state_hash = H(  
    snapshot_schema_version,  
    portfolio_state, slot_state, security_state,  
    pending_actions, ledger_state, sentinel_state)
```

Absent-means-zero is forbidden for behavioural fields. Stocker's

`PortfolioState.from_dict` currently defaults missing fields to zero, justified by a comment saying Wealth Core has never run in production. That justification expires the moment snapshots become operational truth.

Field taxonomy for migrations:

STATIC / intrinsic	serialise and validate directly	
DERIVED / replayable	may be rebuilt from pinned history	
EVENT-MEMORY	observable only once, in the past;	
MUST be serialised	or recovered from an earlier	
authoritative anchor		

Worked example from the C1 settlement work:

sessions_since_valid_mark marks	DERIVED	– rebuildable from	
terminal_pending_sessions appeared on ONE	EVENT-MEMORY	– the ACTIONS row	
terminal_pending_terms again	EVENT-MEMORY	session and never	

So a v1 -> v2 migration is **conditional**: safe when nothing is mid-grace, otherwise it must FAIL and say "restore from an earlier trusted anchor". And "nothing is mid-grace" may not be inferred from the absence of the fields.

A migration is allowed to fail. Silent reinterpretation is not.

🔗 6. Versioning and fingerprints

```
strategy_fingerprint = SHA256(
    wealth_core_config_hash
    sentinel_config_hash
    ordering_profile_hash
    execution_model_hash
    data_contract_version
    golden_fixture_hash)      <- BEHAVIOURAL identity, see
below

provenance (recorded, NOT in the fingerprint):
    git_sha
    image_digest
    build_timestamp

recorded on every decision, alongside the fingerprint:
    data_version              <- which corpus the rules
were evaluated against
    snapshot_schema_version
    snapshot_hash
    prior_decision_hash
```

Why the golden fixture hash and not a code hash. A code SHA answers "what source built this binary?"; the golden hash answers "what semantics does this engine exhibit?". Certification cares about the second. In Stocker the golden hash moved when the C1 grace period changed semantics and did **not** move when streaming hashes, settlement counters or retention bounding were added — exactly the discrimination wanted. A release can then say:

```
git_sha changed, image_digest changed, behavioural hash
unchanged
=> implementation changed; certified behaviour did not
```

data_version is deliberately outside the fingerprint. The strategy did not change when the vendor corrected a historical record. Keeping them separate lets you answer two different questions: which rules produced this decision, and against which corpus were they evaluated.

🔗 **PREREQUISITE: the golden fixture is not yet a complete semantic discriminator**

Do not put the golden hash into the fingerprint until this is fixed. Three demonstrated gaps in Stocker:

- 1 the terms-block path is no longer exercised at all – SEC_STRANDED was its only case and now correctly CARRIES instead
- 2 the grace-clock defect was invisible to it – SEC_STRANDED keeps printing and its terms arrive on session 10 of a 10-session grace, so the counter reached 9 and never expired. The hash would not have moved had the clock been wrong
- 3 the settlement counters are deliberately excluded from the hash

🔗 Semantic coverage must be **OBSERVED** from execution, not inferred from scenario

Stocker already names its scenario constants and asserts they are referenced. That was not enough: the scenario did not change, the **semantics** did.

`STRANDED_ANNOUNCED` still fires and still has an assertion, but what it proves changed from "block" to "carry". Coverage was lost with no constant going unreferenced.

So the **engine** must emit branch-level markers, and CI must gate on them:

```
semantic_branch IDs (coarse, stable, versioned – NOT diagnostic  
prose):  
  terminal_announced, terminal_carry, terminal_grace_expired,  
  terminal_terms_block, terminal_settled, write_off,  
conversion_applied,  
  split_applied, dividend_applied, stop_triggered,  
cooldown_entered,  
  review_fired, ... plus every Sentinel transition
```

The manifest has two sections with different force:

```

required_semantic_surfaces:    NORMATIVE – must be exercised
every release by
  terminal_terms_block: PASS    the synthetic/golden suite,
regardless of which
  terminal_grace_expiry: PASS    historical window is being
certified
...

corpus_observed_surfaces:      DESCRIPTIVE – evidence
quality, never a
  terminal_terms_block: 0        substitute for the floor. A
quiet window must
  terminal_settle: 11            not lower the bar; it says so
in its manifest
  stop_triggered: 384

```

Fixture and corpus are near-disjoint on settlement, which is why both are needed. Sharadar states no per-share consideration for any of its 19,216 delisted securities, so `EXACT_TERMS` is structurally unreachable from the corpus — while it is the branch the golden fixture exercises most. The corpus instead proves the carry -> derived-last-mark path at a scale no fixture can manufacture.

```

synthetic fixture    BRANCH COMPLETENESS
historical corpus     PRODUCTION-SHAPED STRESS AND FREQUENCY

```

🔗 Namespaces that must never share keys

Stocker had a real bug here: `SettlementDecision.provenance()` carried a bare `reason` key that dict-spread let the caller's block reason clobber, so the audit said `NO_TRUSTWORTHY_MARK` where `MISSING_CASH_PER_SHARE` belonged. Two questions, one key, resolved silently by insertion order.

```

semantic_branch    what behaviour occurred    TERMINAL_BLOCK
decision_reason    why the engine chose it
MISSING_CASH_PER_SHARE
evidence           what facts supported it
{last_valid_mark_reason: ...}

```

Typed objects, not free-form dicts.

7. Sentinel 1.1 — the controller

Source of truth is the frozen research harness, NOT this document. The numbers below are transcribed from the prospectus PDFs and are here for orientation. They must be imported as explicit versioned configuration from the harness code before implementation. Re-inferring Sentinel from prose is how you get a Sentinel that merely resembles 1.1.

States

```
NORMAL                live = 100% Wealth Core
ORDINARY_STRESS       live = 100% Wealth Core  <- a SENSOR, not
an actuator
SEVERE_STRESS_FAST    live = 0%, defensive sleeve = 100%
SEVERE_STRESS_SLOW    live = 0%, defensive sleeve = 100%
```

Unlike earlier versions, **ordinary stress does not reduce exposure**. Sentinel 1.1 stays invested through ordinary corrections. Ordinary stress exists so the slow path can ask whether Wealth Core has been impaired long enough; its trigger is shadow drawdown crossing about **-15.5%**, which starts/maintains the stress clock and changes no allocation.

Fast path — sudden systemic crash

All of the following, simultaneously:

```
shadow drawdown        <= -10%
damaged breadth        >= 85%
green breadth          <= 20%
AND (shadow r5 <= -5%  OR  shadow r10 <= -8%)
damaged breadth increased >= 40 percentage points over 5
sessions
SPY vol15 / vol20 - 1   >= 4%
AND (SPY r20 <= -1%    OR  shadow r10 <= -10%)
```

Fast recovery: minimum severe hold of ~10 sessions, then **three consecutive** healthy closes with `shadow r20 > 0`, `damaged breadth <= 60%`, `green breadth >= 20%`. The detector re-arms only after shadow drawdown improves above roughly -6% and the shock condition is absent.

⌘ Slow path — grinding bear

Evaluated only while ordinary stress is active:

```
ordinary stress active
stress age                >= 30 sessions
shadow return since stress start <= -2%
shadow r40                <= -3%
damaged breadth           >= 75%
green breadth             <= 25%
```



(An older research form also required `positive-day share <= 50%`; the simplified version removed it because it changed no historical decision.)

Slow recovery is deliberately more conservative: at least **20 sessions** in the slow severe state, then **five consecutive** healthy closes on the same three conditions.

⌘ Causes are tracked separately; the actuator is trivial

```
fast_severe_active: bool
slow_severe_active: bool
severe = fast_severe_active or slow_severe_active
target_core_exposure = 0.0 if severe else 1.0
```



Not one generic severe flag. The causes can overlap and their recovery semantics differ; if fast clears while slow remains active, exposure stays 0% but the governing recovery clock changes. Reason codes must preserve which cause fired and record cause-set transitions as auditable events even when target exposure is unchanged.

⌘ Evidence records, not boolean expressions

Seven conditions with two embedded disjunctions is where a transcription error hides silently. Each predicate carries value, availability and pass/fail separately:

```
FastSevereEvidence(
    shadow_dd          = PredicateResult(value=...,
available=..., passed=...),
    damaged_breadth    = ...,
    green_breadth      = ...,
    short_loss_r5      = ...,
    short_loss_r10     = ...,
    damage_acceleration = ...,
    vol_acceleration    = ...,
    spy_r20            = ...,
    shadow_r10_fallback = ...,
)
```



`None` means **unavailable** and must stay distinguishable from `False`. A required-but-unavailable predicate fails the transition **closed**, with an explicit reason (`FAST_EVIDENCE_UNAVAILABLE`), never coerced into an ordinary negative.

This is a direct lesson from Stocker's crash brake, which was fail-open on the restore side because one boolean answered both "the evidence says no crash" and "there is no evidence". Both engines must emit the same canonical `transition_record()` / `transition_hash()`; parity is asserted on the decision record, not on the presence of a call site.

🔗 Execution semantics

Decisions are made **after the close**; any exposure change is effective at the **next eligible open**. The overnight gap belongs to the old allocation — Sentinel does not pretend it could have exited at the decision close. Transaction costs are charged only on changed notional. The defensive sleeve is BIL / short-duration Treasury total return, and the research replay must use the frozen proxy consistently.

🔗 Sentinel event-memory state (mandatory in the snapshot, fail closed)

```
ordinary_stress_active
ordinary_stress_start_session
ordinary_stress_start_shadow_nav
ordinary_stress_age
fast_severe_active
fast_severe_entry_session
fast_healthy_streak
fast_rearm_armed
slow_severe_active
slow_severe_entry_session
slow_healthy_streak
slow_stress_reference_session
```



None of these are replayable from a short history window. A silently reset slow-stress clock delays a severe exit by 30 sessions while looking healthy.

🔗 8. OPEN QUESTIONS — resolve before writing code

These are not engineering details. They are semantic forks that determine the architecture, and **none can be answered from this repository**. They require the frozen research harness.

🔗 Q1 (pivotal). Scalar or share-level?

Does the frozen harness maintain an independent share-level live portfolio, or does it compute live performance as an exposure scalar applied to the shadow?

Evidence points at **scalar**: binary target exposure, "costs on changed notional" (scalar language — a share-level book charges per-security costs on actual trades), and an immutable shadow as the sole underlying book.

If scalar, there are no live Wealth Core episodes at all, and Q3 dissolves.

But scalar is a gap, not a simplification. A broker cannot hold "1.0 x shadow". Production needs share-level quantities, next-open fills and reconciliation. That means the production engine **extends** the harness rather than reproducing it, and the certification claim must be worded accordingly (see §9).

Q2. Breadth definitions — the hard blocker

`damaged_breadth` and `green_breadth` appear in every threshold of both paths and **are not defined anywhere in this repository**. They live in the frozen harness.

No Sentinel decision logic may be written until they are imported from the **certified code path and proven against a known breadth tape**. They are part of the strategy definition, not a feature to reproduce later: if their semantics drift, both paths become different strategies even with every threshold transcribed perfectly.

Q3. Recovery episode semantics (moot if Q1 is scalar)

When Sentinel returns to 100%, do live positions inherit shadow episode state?

A fresh live episode	entry price, peak, age, review
clock all reset	
B shadow continuation	inherits peak, age, review state



Wealth Core's own semantics say a re-entry is a **new episode with a fresh peak** — so under A, every defensive round-trip re-arms every trailing stop and resets every 119-session review clock. Materially different from B. Do not choose from first principles.

Harness query: on a recovery date where the shadow holds a security materially below its episode peak, what peak/entry/review state does the newly live position receive?

Q4. How exactly is live NAV computed during and across transitions?

Actual next-open basket values, or a multiplied return series? This determines whether a share-level implementation can reproduce the reference **exactly** or only within a **bounded error**.

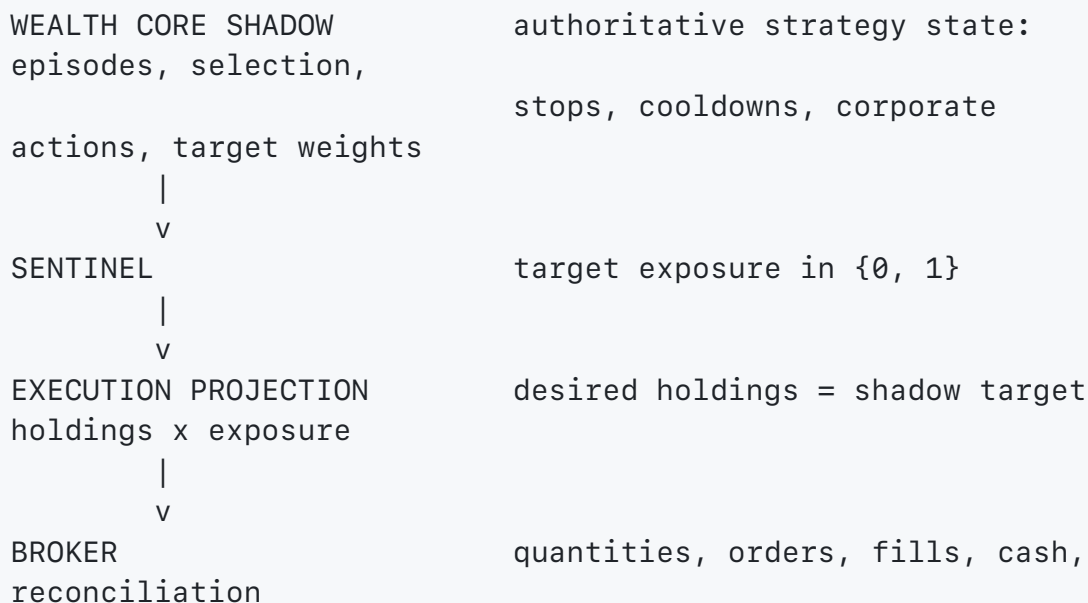
Harness-reading order

- 1 scalar vs independent live book (Q1)
- 2 if scalar: live NAV, BIL return, changed-notional cost computation
- 3 transitions: real next-open basket values or multiplied return series (Q4)
- 4 breadth definitions (Q2)
- 5 recovery episode semantics (Q3)
- 6 THEN decide the production architecture



9. Production architecture if Q1 is scalar (most likely)

Not two Wealth Core books — **one state machine, one controller, one execution projection.**



The live side is **not** a second Wealth Core state machine. Do not duplicate slot state, peaks, review clocks, cooldowns or terminal memory into it. Live state holds only what is needed to execute and reconcile:

broker positions, quantities, cash, pending orders, fills,
mark/NAV, reconciliation status



Worked example. Shadow owns XYZ at 4%, its episode peak 25% above current price. Sentinel goes defensive: live sells XYZ, the shadow keeps its episode running. Sentinel recovers: live buys XYZ again — but this creates **no new Wealth Core episode**. The shadow still considers it the same old episode, and if the shadow's continuing stop fires tomorrow the desired live target drops it too.

🔗 Three exposure quantities, never conflated

sentinel_target_exposure	0.0 or 1.0 – what the certified controller wants
execution_target_exposure	translated into a concrete target basket
realized_exposure	what the broker actually achieved

This state is legitimate and must be representable:

sentinel target	1.00
execution target	1.00
realized	0.96
pending	0.04
execution_status	PARTIAL

Realized exposure must never feed back into Sentinel's state machine. Sentinel judges the immutable shadow and its own event memory. Otherwise an operational fill problem becomes a market-state input.

Execution incompleteness may delay achievement of Sentinel's target, but it must never change Sentinel's target or clear the pending intent.

Recovery flow:

close t	Sentinel decides target = 1.0
open t+1	build orders for the current shadow basket
	fill every executable leg; leave non-fillable legs
PENDING	
after fills	compute realized_exposure; persist the outstanding delta
later opens	retry pending legs under the frozen execution rules

Share rounding and affordability live here too: with 25 names, exact 4% weights are not achievable in integer shares. Persist both target and realized basket weights; leftover cash is execution residual, not a Sentinel decision.

🔗 The exposure gap is directional, and the two directions are opposite failures

```
transition_direction = DEFENSIVE | RECOVERY
target_exposure, realized_exposure, exposure_gap,
pending_notional, pending_leg_count, sessions_since_transition
```

```
RECOVERY    target 1.00, realized 0.96, gap -0.04
             opportunity cost. Underinvested. A footnote.

DEFENSIVE   target 0.00, realized 0.08, gap +0.08
             8% of NAV still exposed after Sentinel declared
             severe stress.
             The controller is correct; execution is incomplete.
             HIGH severity.
```

Partial execution is overwhelmingly a **recovery** phenomenon — liquidations fill wherever there is a print, purchases face affordability, rounding and no-print legs. The dangerous exception is a security that **halts during a crash** and cannot be sold. Stocker's `PendingOrder` already persists intent across non-tradeable sessions for exactly this reason.

Metrics, split by direction:

```
RECOVERY    recovery_time_to_target_sessions,
             recovery_peak_underexposure,
             recovery_pending_notional_peak,
             recovery_unfilled_leg_sessions
DEFENSIVE   defensive_time_to_target_sessions,
             defensive_peak_residual_exposure,
             defensive_residual_notional_peak,
             defensive_unliquidated_leg_sessions
```

Persist offending security IDs and reasons, not just the aggregate gap — "8% residual" does not say whether that is one position or four, or why.

Also persist `residual_exposure_pnl_since_defensive_signal`, so you can say: *Sentinel requested zero exposure on date X; execution could not remove 7.4% of NAV; that residual subsequently cost 1.2% of NAV before liquidation was possible.* That separates strategy drawdown from execution-imposed drawdown.

🔗 10. Certification: two claims, not one

Controller certification (exact):

Given the same immutable shadow observations, production Sentinel emits exactly the same fast/slow state, reason record and binary target exposure as the frozen research harness.

Execution certification (exact or bounded, depending on Q4):

Given those same target exposures and shadow target holdings, the share-level execution projection reproduces the scalar reference within the defined execution and cost model.

Run both simultaneously and decompose any residual into: share rounding, missing/open-price policy, transaction-cost modelling, cash residual, corporate-action timing, unfilled legs. Once proven, the share-level implementation becomes the stronger certification anchor.

If the harness is a pure return-series scalar, call the second claim a **bounded-error execution-equivalence** claim. Forcing a false equality is worse than stating the bound.

🔗 Live performance is then additively decomposable

```
live return = shadow return      strategy    – did Wealth Core work?  
                + Sentinel timing    intent      – did the  
exposure calls help?            attainment – did we achieve  
                + execution slippage them?
```

Every term is already computed: the shadow is authoritative and continuous, the scalar reference (shadow when 1.0, BIL when 0.0) is the counterfactual for the middle term, and the difference between reference and realized is the third.

This is what makes Sentinel 1.2 evaluable — identical shadow, identical execution model, different controller, and the middle term is the whole comparison. It is also what answers "was that the strategy, the controller, or the plumbing?" after a bad quarter. A design that cannot separate those invites the wrong fix.

🔗 11. Implementation order

Steps 0-2 are blocking. Do not reorder them.

0 Locate and attach the frozen research harness. Sentinel 1.1 is NOT fully specifiable from this repository alone.

1 Answer Q1 and Q4 (scalar vs share-level; how live NAV is computed).
This determines the whole production architecture.

2 Import and PROVE breadth semantics (Q2) against a known frozen tape.
No decision logic before this.

3 Answer Q3 if Q1 turns out to be share-level.

4 Introduce the shadow/live split appropriate to the Q1 answer.

5 Extend snapshot/restart to every new object: shadow state, shadow ledger,
live/execution state, Sentinel controller state, pending actions.

6 Shadow-independence falsifier: run with Sentinel disabled vs Sentinel at 0%
for an interval; assert shadow state, NAV, holdings, peaks, slot state,
cooldowns, review state, terminal state and decision hashes are IDENTICAL.

7 Sentinel event-memory snapshot schema, fail-closed restore.

8 Typed fast/slow evidence records; no None -> False coercion.

9 Canonical transition records and hashes, emitted identically in both roles.

10 Fast and slow state machines, independently.

11 Trivial actuator: severe = fast or slow; exposure = 0 or 1.

12 Execution projection and next-open execution, live side only.

13 Extend the restart matrix across the new seams.



Restart matrix cases the new seams require

fast severe active
slow severe active
both active
pending transition at the boundary
healthy streak mid-count
ordinary stress clock mid-run
restart while live and shadow differ



Plus the case that connects Sentinel to the settlement work:

```
Sentinel at 0% live exposure
shadow holds SEC_X
SEC_X has terminal_pending_terms / a grace in progress
-> restart
-> shadow event state identical
-> grace counter does NOT reset
-> live remains empty
-> subsequent settlement outcome identical to an
uninterrupted run
```



Shadow state persistence is independent of live ownership. At 0% exposure the shadow holds everything and live holds nothing, so terminal terms, grace counters, last-valid-mark state, conversion state, dividend receivables and pending settlements all live in the shadow with no live counterpart. A restore implementation must never derive "which securities need persisted event memory" from live holdings.

That bug would stay invisible for many sessions and surface only when the terminal branch finally resolved.

🔗 12. The invariants, collected



- 1 Same image, different roles. sentinel-live and sentinel-bt run the same
sentinel-engine:<git-sha> artifact.
- 2 Behavioural identity, not source identity. The golden fixture hash
participates in the strategy fingerprint; git SHA is provenance only –
AFTER the fixture is proven a complete semantic discriminator.
- 3 Pinned input history. Every snapshot and decision records data_version.
- 4 Versioned state semantics. snapshot_schema_version is inside the state hash;
behavioural fields fail closed, never default.
- 5 Restart equivalence. restore + prime + run_until is hash-equivalent to
uninterrupted execution.
- 6 No mutable shared Feed across replay seams.
- 7 Live recovery is snapshot + short replay. Full-inception reconstruction is
an audit operation, not a boot path.
- 8 The shadow never de-risks and is never derived from the live book.
- 9 Sentinel scales HOW MUCH of the target is held, never WHAT is in it.
- 10 Realized exposure never feeds back into the controller.
- 11 Execution incompleteness delays attainment; it never changes intent.
- 12 Fast and slow severe causes are tracked separately and permanently.
- 13 Close-t decision executes at open-t+1; the overnight gap belongs to the old
allocation.
- 14 Semantic coverage is observed from execution, never inferred from scenario
construction. Fixture coverage is normative; corpus coverage is descriptive.

Nearly all of these are falsifiable. Each should have a test that fails when it is violated, and #9, #10 and #11 in particular are the ones a well-meaning refactor is most likely to break silently.

